

# Verity - Citation-Grounded Support Triage Copilot

**Author:** Royce Barboz

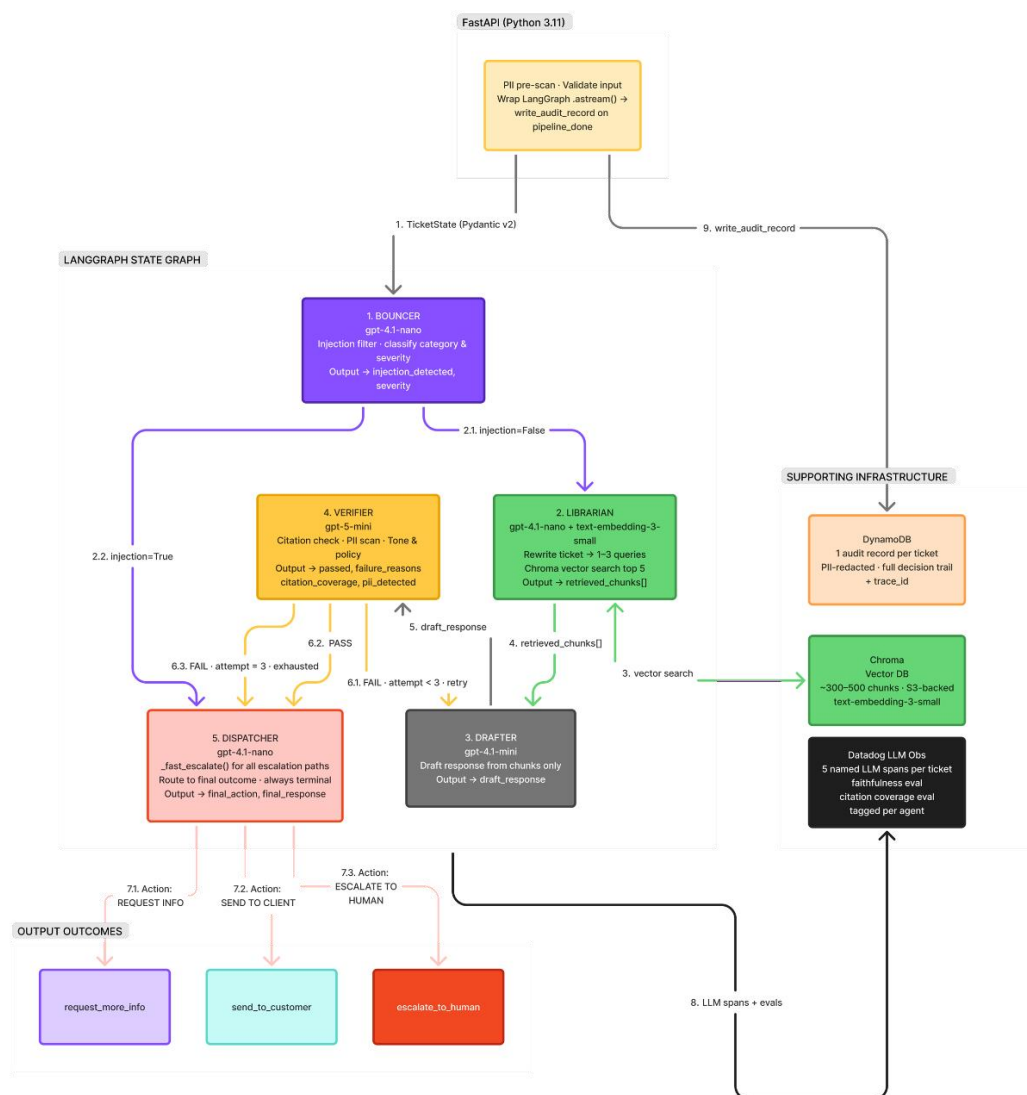
**Date:** 2026-05-19

**Repository:** <https://github.com/roycebarboz/Verity>

**Live demo:** <http://verity-alb-2072832666.us-east-1.elb.amazonaws.com/>

Verity is a five-agent system that automates customer-support ticket triage while guaranteeing that every customer-facing reply is grounded in retrieved internal documentation. The design deliberately separates drafting from verification across distinct models, so no uncited claim, PII leak, or prompt-injection bypass reaches a customer. Verity targets enterprise support operations where the cost of a wrong AI response (refund commitments, policy misstatements, regulatory leaks) outweighs the cost of automation itself.

## 1. Multi-Agent Architecture



Verity is a sequential pipeline of five specialized agents, each with a single responsibility, a single structured-output schema, and a single model tier:

| # | Agent      | Role                                                | Model                                 |
|---|------------|-----------------------------------------------------|---------------------------------------|
| 1 | Bouncer    | Classify category/severity; detect prompt injection | gpt-4.1-nano                          |
| 2 | Librarian  | Rewrite ticket → 1–3 queries; retrieve top-5 chunks | gpt-4.1-nano + text-embedding-3-small |
| 3 | Drafter    | Generate grounded reply from retrieved chunks only  | gpt-4.1-mini                          |
| 4 | Verifier   | Citation, PII, tone, and policy check               | gpt-5-mini                            |
| 5 | Dispatcher | Route to send / escalate / request_info             | gpt-4.1-nano                          |

**Communication.** Agents exchange data exclusively through a typed Pydantic v2 object (TicketState) passed through a LangGraph state machine — no free-form chat, no shared scratchpad. Every output is schema-validated at the boundary before being merged into state.

**Topology.** Sequential with one conditional retry loop and two hard-escalation exits. The Bouncer escalates immediately on detected injection; the Verifier escalates after two failed Drafter retries (three attempts total). The Drafter↔Verifier loop is the only feedback path — Verifier failure reasons are appended to the next Drafter prompt so the retry is informed, not blind.

**Justification.** Concentrating reasoning capacity at the Verifier (where a missed hallucination is most costly) and stripping the Drafter of all tools and external access creates defense in depth at the model-architecture level. A compromised or hallucinating Drafter cannot exfiltrate data or invent policy — the Verifier is structurally incapable of trusting its output.

## 2. Security, Safety, and Guardrails

**Input boundary.** Every payload is length-capped at 4,000 characters and schema-validated by FastAPI before any LLM is invoked. A 15-pattern regex pre-filter in [backend/verity/guardrails.py](#) catches well-known injection idioms (ignore previous instructions, ===SYSTEM===, [INST], DAN/jailbreak templates, act as...) cheaply; the Bouncer then runs an LLM-based classifier as a second layer. Tickets flagged as injection bypass the rest of the pipeline and route directly to human escalation, with the original input quoted but never executed.

**Output boundary.** The Verifier enforces three hardcoded gates before any response can be sent: a PII regex suite (email, phone, SSN, credit card, AWS keys), a citation-coverage check (3-

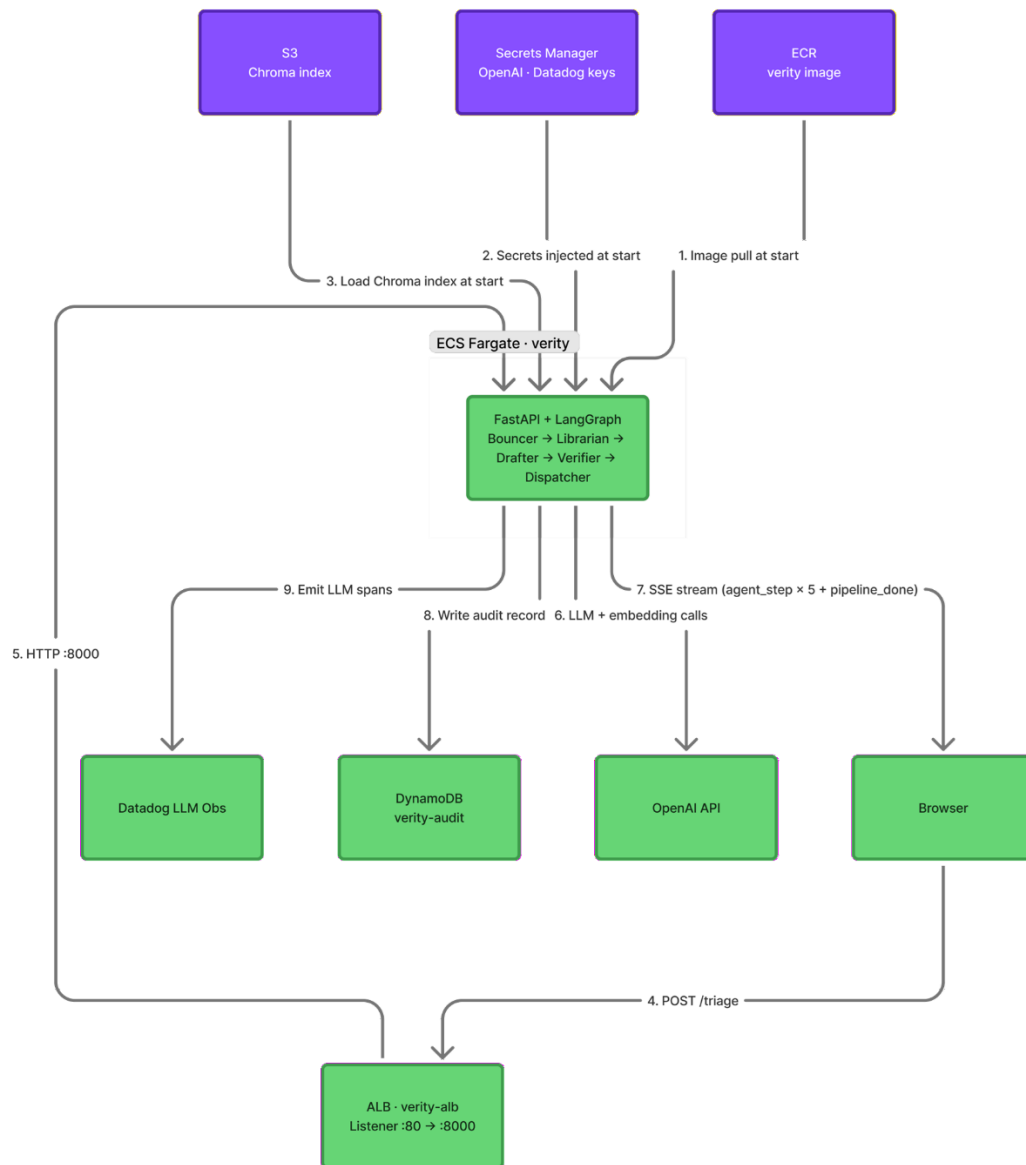
gram overlap between draft claims and retrieved chunks), and a tone/policy check. Two failed verifications force escalation.

**Data handling.** PII is redacted before any persistent write to DynamoDB; raw text fields are stored redacted, and customer\_id is hashed. The full pipeline decision trail and the Datadog trace ID are stored under one ticket\_id, giving compliance a queryable audit record per ticket.

**Secrets and IAM.** OpenAI and Datadog API keys live in AWS Secrets Manager and are injected at container start — they never appear in code, logs, or Terraform state. The Fargate task IAM role grants exactly two secret reads, one dynamodb:PutItem, and one s3:GetObject on the Chroma bucket — nothing else. The task security group accepts ingress only from the ALB; the ALB is the sole public surface.

**Containment.** No agent has shell, network, or arbitrary DB access. The Drafter in particular is configured with zero tools — it can only read the chunks it is handed. Any unhandled exception short-circuits to a deterministic escalate\_to\_human outcome rather than retrying blindly.

### 3. Implementation Approach



**Stack.** Python 3.11 + FastAPI for the API layer, LangGraph for the state machine, Pydantic v2 for schemas, Chroma (embedded) for the vector index, OpenAI for inference, Datadog LLM Observability for traces, DynamoDB for the audit log, and ECS Fargate + an Application Load Balancer for hosting. Infrastructure is one-shot deployable via Terraform ([infra/main.tf](https://github.com/verity-ai/infra/main.tf)).

**Frontend.** Vite + React + TypeScript + Tailwind, compiled to static assets and served from the same FastAPI container — one origin, no CORS attack surface in production. The UI consumes Server-Sent Events from POST /triage: each agent card reveals as its agent\_step event arrives, and the final outcome populates on pipeline\_done.

**Agent lifecycle.** Each agent is a node in the LangGraph compiled graph ([backend/verity/graph.py](https://github.com/verity-ai/backend/verity/graph.py)). LangGraph handles instantiation, state propagation, and

termination; the API layer ([backend/verity/api.py](#)) wraps the graph in astream, emits SSE events as nodes complete, and writes the audit record on pipeline\_done.

**Error handling.** OpenAI calls use exponential backoff with jitter (3 attempts on 429/5xx). Schema-parse failures retry the same agent up to two times with the parse error appended to the prompt. The Drafter↔Verifier loop is bounded at three attempts. A per-request 8,000-token hard cap prevents runaway loops. Any unhandled exception escalates deterministically.

**Testing.** A 30-ticket offline eval suite ([scripts/run\\_eval.py](#)) measures pass rate, citation coverage, retrieval precision@5, PII leak count, and injection block rate against the live endpoint. A stress-test harness ([scripts/stress\\_test.py](#)) covers injection, PII, retry-loop, compliance, and boundary cases with color-coded output.

## 4. Use of AI / LLMs and Collaboration

**Tiered model selection.** Reasoning capacity is concentrated where it matters most. The Bouncer, Librarian, and Dispatcher use gpt-4.1-nano — the cheapest tier — because their tasks are deterministic classification or routing. The Drafter uses gpt-4.1-mini for fluent customer-facing prose without paying for reasoning it doesn't need. Only the Verifier uses the reasoning tier (gpt-5-mini), since the cost of a missed hallucination is asymmetric: every other agent's mistake can be caught, but a Verifier false-pass goes straight to the customer.

**Embedding choice.** text-embedding-3-small (1536-dim) over 3-large (3072-dim) because at demo-scale corpus size (60 documents, ~300–500 chunks) retrieval quality is bottlenecked by corpus design, not embedding dimensionality — the larger model would double index cost with no measurable precision gain.

**Collaboration pattern.** Structured handoff via shared state — no negotiation, no consensus, no free-form chat. The Verifier acts as a critic with veto power over the Drafter, and failure reasons are appended verbatim to the Drafter's next prompt so the retry is informed. This is *adversarial collaboration* in the GAN sense: the Drafter's incentive is fluency, the Verifier's is strictness, and the system's correctness emerges from their disagreement.

**Autonomy vs. control.** Verity deliberately chooses control over autonomy. Small models for cheap tasks, no tools for the Drafter, exactly three terminal actions (send / escalate / request\_info), and an audit record per ticket. The cost is rigidity — Verity cannot improvise outside its prescribed actions. The benefit is that a compliance team can approve it for production: every decision is reviewable in DynamoDB, linked to a Datadog trace with five named LLM spans, and every Drafter span carries faithfulness and citation-coverage evaluations.

## Evaluation Results

| Metric                       | Target      | Result               |
|------------------------------|-------------|----------------------|
| Pipeline pass rate           | $\geq 60\%$ | <b>76.7% (23/30)</b> |
| Mean citation coverage       | $\geq 0.85$ | <b>0.68</b>          |
| PII leaks (sent responses)   | 0           | <b>0</b>             |
| Successful prompt injections | 0           | <b>3/3 blocked</b>   |
| Retrieval precision@5        | $\geq 0.7$  | <b>0.72</b>          |
| End-to-end P50 latency       | $\leq 30$ s | <b>25.4 s</b>        |